Study and Implementation of Python Libraries for ML (NumPy, Pandas, Matplotlib)

```
# Step 1: Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Step 2: NumPy operations
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr.shape, arr.reshape(3, 2).shape)      # shape and reshape
print(arr[1, 2], arr[0, :])                    # indexing and slicing
print(np.sum(arr), np.mean(arr), arr + 10)     # arithmetic

# Step 3: Create a Pandas DataFrame
df = pd.DataFrame({"Name": ["Amit", "Priya", "Rahul", "Sneha", "Karan"],
                   "Marks": [85, 92, 78, 88, 95],
                   "Subject": ["ML", "ML", "DBMS", "ML", "DBMS"]})

# Step 4: Basic data analysis
print(df.head())
df.info()
print(df.describe())
print(df.isnull().sum())

# Step 5: Filter, group and sort
print(df.loc[df["Subject"] == "ML"])
print(df.groupby("Subject")["Marks"].mean())
print(df.sort_values("Marks", ascending=False))

# Step 6: Visualize - line plot, bar chart, histogram
plt.plot(df["Name"], df["Marks"]); plt.show()    # line plot
plt.bar(df["Name"], df["Marks"]); plt.show()     # bar chart
plt.hist(df["Marks"]); plt.show()                # histogram
```

Implementation of the Complete Machine Learning Workflow using Scikit-learn

```
# Step 1: Import libraries
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 2: Load dataset
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name="species")

# Step 3: Explore dataset
print(X.head())
X.info()
print("Missing values:\n", X.isnull().sum())

# Step 4: Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Step 5: Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 6: Train the model
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

# Step 7: Predict on test data
y_pred = model.predict(X_test)

# Step 8: Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

Analysis of Bias, Variance, Overfitting and Underfitting using Polynomial Regression

```
# Step 1: Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error

# Step 2: Generate non-linear data with noise
np.random.seed(42)
X = np.sort(np.random.rand(60, 1) * 6 - 3, axis=0)
y = 0.5 * X[:, 0]**2 + X[:, 0] + 2 + np.random.randn(60) * 1.5

# Step 3: Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
plt.scatter(X_train, y_train)
plt.scatter(X_test, y_test)
xs = np.linspace(-3, 3, 200).reshape(-1, 1)

for d in [1, 2, 15]:
    # Step 4 & 5: Polynomial features + Linear Regression
    model = make_pipeline(PolynomialFeatures(d), LinearRegression())
    model.fit(X_train, y_train)

    # Step 6: Predict and compute MSE
    train_mse = mean_squared_error(y_train, model.predict(X_train))
    test_mse = mean_squared_error(y_test, model.predict(X_test))
    print(f"Degree {d}: Train MSE = {train_mse:.3f}, Test MSE = {test_mse:.3f}")

    # Step 7: Plot fitted curve for each degree
    plt.plot(xs, model.predict(xs), label=f"Degree {d}")

plt.ylim(-5, 15)
plt.legend()
plt.show()

# Step 8: Compare errors
# Degree 1  : both errors high           -> Underfitting (high bias)
# Degree 2  : both errors low and close  -> Good fit
# Degree 15 : low train, high test error -> Overfitting (high variance)
```

Implementation of Simple Linear Regression for Predicting a Continuous Variable

```
# Step 1: Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Step 2: Create dataset (Years of Experience vs Salary)
np.random.seed(1)
X = np.random.uniform(1, 10, (50, 1))
y = 25000 + X[:, 0] * 8000 + np.random.randn(50) * 5000

# Step 3: Scatter plot
plt.scatter(X, y)
plt.show()

# Step 4: Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 5: Train the model
model = LinearRegression()
model.fit(X_train, y_train)
print("Slope:", model.coef_[0], "Intercept:", model.intercept_)

# Step 6: Predict
y_pred = model.predict(X_test)

# Step 7: Evaluate
mse = mean_squared_error(y_test, y_pred)
print("MAE :", mean_absolute_error(y_test, y_pred))
print("MSE :", mse)
print("RMSE:", np.sqrt(mse))
print("R2  :", r2_score(y_test, y_pred))

# Step 8: Plot regression line
plt.scatter(X, y)
plt.plot(X, model.predict(X), color="red")
plt.show()
```

Implementation of Multiple Linear Regression using Multiple Features

```
# Step 1: Import libraries and load Diabetes dataset
import numpy as np
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

data = load_diabetes()

# Step 2: Convert to DataFrame and examine
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name="disease_progression")
print(X.head())
print(y.head())

# Step 3: Feature matrix (X) has 10 columns, target vector is y
print("Features shape:", X.shape)

# Step 4: Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Step 5: Create and train the model
model = LinearRegression()
model.fit(X_train, y_train)

# Step 6: Predict on test data
y_pred = model.predict(X_test)

# Step 7: Evaluate using MAE, MSE, RMSE, R2
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print(f"MAE: {mae:.2f}\nMSE: {mse:.2f}\nRMSE: {rmse:.2f}\nR2: {r2:.3f}")

# Step 8: Display learned coefficients
print("Coefficients:\n", pd.Series(model.coef_, index=X.columns))
print("Intercept:", model.intercept_)
```

Implementation and Comparison of Ridge, Lasso and Elastic Net Regression

```
# Step 1: Import libraries and load Diabetes dataset
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.metrics import mean_squared_error, r2_score

data = load_diabetes()
X, y = data.data, data.target

# Step 2: Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Steps 3-6: Define baseline, Ridge, Lasso and Elastic Net models
models = {
    "Linear Regression": LinearRegression(),
    "Ridge (alpha=1.0)": Ridge(alpha=1.0),
    "Lasso (alpha=0.5)": Lasso(alpha=0.5),
    "ElasticNet (alpha=0.5, l1_ratio=0.5)": ElasticNet(alpha=0.5, l1_ratio=0.5),
}

# Step 7: Train, predict and compute RMSE and R2 for each model
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)
    zeros = np.sum(np.isclose(model.coef_, 0))

    # Step 8: Compare coefficients (shrinkage / feature selection)
    print(f"{name}: RMSE={rmse:.2f}, R2={r2:.3f}, Zero coefficients={zeros}")
    print("   Coefficients:", np.round(model.coef_, 1))
```

Implementation of Logistic Regression for Binary Classification

```
# Step 1: Import libraries and load Breast Cancer dataset
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report, ConfusionMatrixDispl

data = load_breast_cancer()

# Step 2: Separate features (X) and binary target (y)
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target          # 0 = malignant, 1 = benign

# Step 3: Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Step 4: Scale the features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 5: Create and train the model
model = LogisticRegression(max_iter=5000)
model.fit(X_train, y_train)

# Step 6: Predict class labels
y_pred = model.predict(X_test)

# Step 7: Evaluate using confusion matrix and accuracy
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=data.target_names))

# Step 8: Visualize confusion matrix as a heatmap
ConfusionMatrixDisplay(cm).plot(cmap="Blues")
plt.show()
```

Implementation of K-Nearest Neighbors (KNN) Classification and Impact of K

```
# Step 1: Import libraries and load Iris dataset
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()

# Step 2: Separate features (X) and target (y)
X, y = iris.data, iris.target

# Step 3: Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

# Step 4: Scale features (KNN is distance based)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 5 & 6: Train KNN for K = 1 to 20 and record accuracy
k_values = range(1, 21)
accuracies = []
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    accuracies.append(acc)
    print(f"K = {k:2d} -> Accuracy = {acc:.3f}")

# Step 7: Plot accuracy vs K
plt.plot(k_values, accuracies, marker="o")
plt.xlabel("K")
plt.ylabel("Accuracy")
plt.show()

# Step 8: Find the best K
best_k = k_values[int(np.argmax(accuracies))]
print("Best K =", best_k, "with accuracy =", max(accuracies))
```

Implementation of Decision Tree Classifier and Effect of Tree Depth

```
# Step 1: Import libraries and load Breast Cancer dataset
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score

data = load_breast_cancer()

# Step 2: Separate features (X) and target (y)
X, y = data.data, data.target

# Step 3: Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

# Step 4 & 5: Train for max_depth 1 to 15 and record accuracies
depths = range(1, 16)
train_acc, test_acc = [], []
for d in depths:
    tree = DecisionTreeClassifier(max_depth=d, random_state=42)
    tree.fit(X_train, y_train)
    train_acc.append(accuracy_score(y_train, tree.predict(X_train)))
    test_acc.append(accuracy_score(y_test, tree.predict(X_test)))
    print(f"Depth {d:2d}: Train = {train_acc[-1]:.3f}, Test = {test_acc[-1]:.3f}")

# Step 6: Plot training and testing accuracy vs depth
plt.plot(depths, train_acc, label="Train")
plt.plot(depths, test_acc, label="Test")
plt.xlabel("Depth")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# Step 7: Overfitting point = depth where test accuracy stops improving
# while training accuracy keeps increasing (see the plot / printed values)

# Step 8: Visualize a shallow tree
tree3 = DecisionTreeClassifier(max_depth=3, random_state=42)
tree3.fit(X_train, y_train)
plt.figure(figsize=(14, 7))
plot_tree(tree3, feature_names=data.feature_names, filled=True)
plt.show()
```

Implementation of Random Forest and Comparison with Decision Tree Classifier

```
# Step 1: Import libraries and load Breast Cancer dataset
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X, y = data.data, data.target

# Step 2: Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

# Step 3: Train a single Decision Tree
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
dt_acc = accuracy_score(y_test, dt.predict(X_test))

# Step 4: Train a Random Forest with 200 trees
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)
rf_acc = accuracy_score(y_test, rf.predict(X_test))

# Step 5: Compare test accuracy
print("Decision Tree Test Accuracy :", round(dt_acc, 3))
print("Random Forest Test Accuracy :", round(rf_acc, 3))

# Step 6: Plot top 10 feature importances from Random Forest
pd.Series(rf.feature_importances_, index=data.feature_names).nlargest(10).plot(kind="barh")
plt.show()

# Step 7: Compare 5-fold cross-validation scores
dt_cv = cross_val_score(dt, X, y, cv=5).mean()
rf_cv = cross_val_score(rf, X, y, cv=5).mean()
print("Decision Tree CV Accuracy :", round(dt_cv, 3))
print("Random Forest CV Accuracy :", round(rf_cv, 3))
```

Implementation of K-Means Clustering and Elbow Method

```
# Step 1: Import libraries and generate unlabeled data
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.8, random_state=42)

# Step 2: Standardize the features
X = StandardScaler().fit_transform(X)

# Step 3: Run K-Means for K = 1 to 10 and record WCSS (inertia)
wcss = []
K_range = range(1, 11)
for k in K_range:
    kmeans = KMeans(n_clusters=k, n_init=10, random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)
    print(f"K = {k}: WCSS = {kmeans.inertia_:.2f}")

# Step 4: Plot the Elbow curve
plt.plot(K_range, wcss, marker="o")
plt.xlabel("K")
plt.ylabel("WCSS")
plt.show()

# Step 5: Identify the elbow point (where the curve bends) -> K = 4
optimal_k = 4

# Step 6: Fit the final K-Means model
kmeans = KMeans(n_clusters=optimal_k, n_init=10, random_state=42)
labels = kmeans.fit_predict(X)

# Step 7: Visualize clusters and centroids
plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], c="red", marker="X", s=200)
plt.show()
```

Implementation of Hierarchical (Agglomerative) Clustering and Dendrogram

```
# Step 1: Import libraries
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

# Step 2: Generate unlabeled dataset
X, _ = make_blobs(n_samples=60, centers=3, cluster_std=0.9, random_state=42)

# Step 3: Compute linkage matrix using Ward linkage
linked = linkage(X, method="ward")

# Step 4: Plot the dendrogram
dendrogram(linked)
plt.axhline(y=10, color="red", linestyle="--")    # cut-off line
plt.show()

# Step 5: Choose number of clusters by counting the vertical lines
#         that the red cut-off line crosses -> 3 clusters
n_clusters = 3

# Step 6: Fit Agglomerative Clustering
model = AgglomerativeClustering(n_clusters=n_clusters, linkage="ward")
labels = model.fit_predict(X)
print("Cluster labels:\n", labels)

# Step 7: Visualize the clusters
plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.show()
```

Implementation of DBSCAN Clustering for Cluster Identification and Outlier Detection

```
# Step 1: Import libraries and generate non-spherical data with noise
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN, KMeans

X, _ = make_moons(n_samples=300, noise=0.08, random_state=42)

# Step 2: Scale the features
X = StandardScaler().fit_transform(X)

# Step 3: Choose eps and min_samples
eps = 0.25
min_samples = 5

# Step 4: Fit DBSCAN
dbscan = DBSCAN(eps=eps, min_samples=min_samples)
labels_db = dbscan.fit_predict(X)

# Step 5: Count clusters and noise points (label = -1)
n_clusters = len(set(labels_db)) - (1 if -1 in labels_db else 0)
n_noise = list(labels_db).count(-1)
print("Clusters found:", n_clusters)
print("Noise points  :", n_noise)

# Step 6: Visualize DBSCAN result (noise points have label -1)
plt.subplot(1, 2, 1)
plt.scatter(X[:, 0], X[:, 1], c=labels_db)
plt.title("DBSCAN")

# Step 7: Compare with K-Means on the same data
labels_km = KMeans(n_clusters=2, n_init=10, random_state=42).fit_predict(X)
plt.subplot(1, 2, 2)
plt.scatter(X[:, 0], X[:, 1], c=labels_km)
plt.title("K-Means")
plt.show()
```

Application of Principal Component Analysis (PCA) for Dimensionality Reduction

```
# Step 1: Import libraries and load Wine dataset
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

data = load_wine()

# Step 2: Separate features (X) and labels (y, only for coloring the plot)
X, y = data.data, data.target
print("Original shape:", X.shape)

# Step 3: Standardize the features
X_scaled = StandardScaler().fit_transform(X)

# Step 4: Apply PCA keeping all components
pca_full = PCA()
pca_full.fit(X_scaled)

# Step 5: Examine explained variance ratio
explained = pca_full.explained_variance_ratio_
cumulative = np.cumsum(explained)
print("Explained variance ratio:\n", np.round(explained, 3))
print("Cumulative variance:\n", np.round(cumulative, 3))

# Step 6: Plot cumulative explained variance
plt.plot(range(1, 14), cumulative, marker="o")
plt.axhline(y=0.90, color="red", linestyle="--")
plt.xlabel("Components")
plt.ylabel("Cumulative Variance")
plt.show()

# Step 7: Reduce to 2 components and visualize
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Reduced shape:", X_pca.shape)

plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```

Hyperparameter Tuning using Grid Search and Cross-Validation

```
# Step 1: Import libraries and load Iris dataset
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

iris = load_iris()
X, y = iris.data, iris.target

# Step 2: Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Step 3: Define the model to be tuned
model = SVC()

# Step 4: Define the parameter grid
param_grid = {
    "C": [0.1, 1, 10, 100],
    "kernel": ["linear", "rbf"],
    "gamma": ["scale", "auto"]
}

# Step 5: Create GridSearchCV with 5-fold cross-validation
grid_search = GridSearchCV(model, param_grid, cv=5, scoring="accuracy")

# Step 6: Fit on training data (searches all combinations)
grid_search.fit(X_train, y_train)

# Step 7: Best hyperparameters and best cross-validation score
print("Best Parameters:", grid_search.best_params_)
print("Best CV Accuracy:", round(grid_search.best_score_, 3))

# Step 8: Evaluate best model on the test set and compare with default SVC
best_acc = accuracy_score(y_test, grid_search.best_estimator_.predict(X_test))
default_acc = accuracy_score(y_test, SVC().fit(X_train, y_train).predict(X_test))
print("Test Accuracy (tuned)  :", round(best_acc, 3))
print("Test Accuracy (default):", round(default_acc, 3))
```